
Report App - Technical Documentation

Project Overview

Report App is a cross-platform mobile and web application designed to streamline report generation, viewing, and management for individuals and organizations. It provides a secure and user-friendly interface for accessing daily reports, filtering by date or category, and exporting reports in multiple formats.

The app supports offline access and ensures data integrity through JWT-based authentication, biometric login, and persistent session management. With responsive UI, light/dark themes, and RESTful API integration, Report App is optimized for productivity, reliability, and ease of use.

Key Functions

Authentication

- Secure login/logout with encrypted credentials
- Optional biometric login (e.g., fingerprint or Face ID)
- Auto session management with refresh tokens
- Secure token storage using SharedPreferences

Report Management

- View daily and historical reports
- Filter reports by date, category, or status
- Search and access detailed report views
- Generate and download reports in PDF, Excel, or CSV formats
- Share reports externally
- Offline caching for access without internet

User Interface

- Clean, intuitive UI with light and dark theme support
- Responsive layout optimized for mobile and web

- Pull-to-refresh for real-time updates
- Loading states and error handling for smooth user experience

Dashboard

- Get insights on total and pending reports
- Track recent report activity
- Personalized dashboard based on user profile

User Management

- View and update user profile
- Manage user preferences
- Role-based access for future scalability

Quick Start Guide

- **Flutter Version: 3.7.2 or higher**
- **Steps:**
 1. **Install Flutter**
 2. **Clone repo**
 3. **Run flutter pub get**
 4. **Configure environment variables**
 5. **Run app using flutter run**

What's Included

- **Cross-platform support: iOS, Android, and Web**
 - **Offline-first functionality**
 - **Full report management system**
 - **User authentication**
-

Core Technologies

Tech	Description
Flutter 3.7.2	UI Framework
Provider 6.1.4	State management
JWT	Secure token auth
REST API	Backend communication
SharedPreferences	Local data persistence

Project Structure

lib/

— main.dart	# App entry point
— models/	# Data models
— screens/	# All UI screens
— widgets/	# Reusable UI components
— services/	# API calls and business logic
— utils/	# Helper methods
— constants/	# Static constants (e.g., colors, strings)

Key Features

Authentication

- Login/logout
- Biometric auth
- Session management
- Token storage

Reports

- View/filter/search
- Share/download reports
- Offline access

UI

- Light/dark themes
 - Responsive layout
 - Pull-to-refresh, error handling
-

API Reference

Authentication

```
class AuthAPI {  
  
    static Future<LoginResponse> login(String username, String password) { ... }  
  
    static Future<void> logout() { ... }  
  
}
```

Reports

```
class ReportAPI {  
  
    static Future<List<Report>> getDailyReports(DateTime date) { ... }  
  
    static Future<Report> getReport(String id) { ... }  
  
}
```

API Endpoints

Authentication

- POST /api/auth/login
- POST /api/auth/refresh

- **POST /api/auth/logout**

User Management

- **GET /api/users/profile**
- **PUT /api/users/profile**

Reports

- **GET /api/reports**
- **GET /api/reports/{id}**
- **POST /api/reports/generate**
- **POST /api/reports/filter**

Dashboard

- **GET /api/dashboard/stats**
-

API Response Examples

Success

```
{  
  "status": "success",  
  "data": {},  
  "message": "string"  
}
```

Error

```
{  
  "status": "error",  
  "error": {  
    "code": "string",  
    "message": "string",  
    "details": {}  
  }  
}
```

```
}  
}
```

Data Models

Report

```
class Report {  
  
    final String id;  
  
    final String title;  
  
    final DateTime date;  
  
    final String status;  
  
    final String? downloadUrl;  
  
}
```

User

```
class User {  
  
    final String id;  
  
    final String name;  
  
    final String email;  
  
    final List<String> roles;  
  
}
```

Developer Guide

Adding a New Screen

1. Create screen in lib/screens/
2. Add navigation route
3. Hook into state management (e.g., Provider)
4. Write widget/unit tests

Resources

-  [Flutter Docs](#)
 -  [API Docs](#)
 -  Support: support@reportapp.com
-